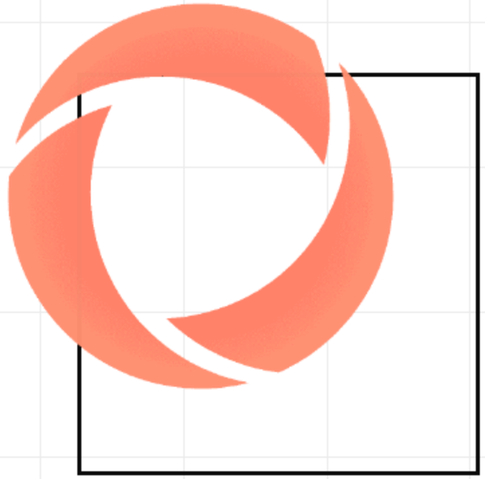

A GUIDED TOUR OF THE FRAMEWORK

Qensei

The AI-assistant-driven QA framework. From a manually-tested ticket to a self-cleaning, REST-first regression pack — with an advisory review panel that raises the floor.

/AUTOMATE • /VALIDATE • /REPORT-BUG

WHAT IT IS

02

A backend-aware QA engine, driven by an AI sensei

Qensei = Q (QA) + sensei. An AI assistant drives it like a QA sensei — **reading** the backend to design cases, **verify** tickets and **diagnose** failures. It analyses and verifies work; it never codes product features. Three capabilities that usually live in separate tools, unified under one product-neutral engine.

Test-case design

Manual-QA case design **with domain knowledge** of the product under test.

REST regression

Automated regression suites run against a live backend — the deterministic gate.

Failure diagnostics

Classify a failure by reading the backend contract, not by guessing.

THE PROBLEM IT SOLVES

Three problems, one design response

03

01

Fragmentation

Manual test design, automated regression, and diagnostics live in three separate tools.

02

Product lock-in

Most QA tooling is coupled to one product. Adding a product means rewriting the tool.

03

The AI-test trap

AI automation passes every mock-level check while silently failing the real boundary.

So — who owns "green" when an AI writes the tests? Qensei answers it structurally: the gate owns green, the human owns convergence, the AI owns implementation.

THE MENTAL MODEL

Three capabilities over one backend connection

04

ONE BACKEND CONNECTION · the SUTConnector

reads the SOURCE (to design + diagnose) and drives the RUNTIME (to execute)

DESIGN

Reads the backend surface — endpoints + business rules — and reports coverage gaps and candidate cases.

`make design`

REGRESS

Runs the pack suite against a live backend. This is the deterministic gate — the source of truth for green.

`make test`

DIAGNOSE

Classifies a failure as REAL_BUG vs TEST_BUG by reading the backend contract.

`make diagnose`

Qensei reads and exercises the backend for QA — to design cases, verify tickets and diagnose failures. It never modifies the product or ships features.

THE DETERMINISTIC CORE

The gate owns “green” — no AI in it

05

The assistant designs, validates, and diagnoses around the gate — it never becomes the gate.

`engine/ · make test · make demo — plain Python 3 stdlib, the single source of truth for green`

Deterministic core

`run.py` runs every pack against the SUT and exits non-zero if any case fails. Zero-dependency. No AI in the loop.

The AI assistant

Slash commands and the review panel work **around** the gate — they accelerate the work, but the human owns convergence.

THE OWNERSHIP MODEL

Who owns what

06

HUMANS	THE AI	THE GATE
OWN INTENT	OWNS THE QA BUILD	OWNS “GREEN”
Intent and design convergence. The spec — the what and the why — is human-approved.	Designs, validates and diagnoses the **tests** around the gate — not product features. The plan — the how — is agent-iterated.	Deterministic, zero-dependency, no AI. It is the only thing that decides pass or fail.

Humans own intent and design convergence; the AI owns implementation.

THE END-TO-END FLOW

From a manually-tested ticket to a pack

07

/validate

Check a ticket's acceptance criteria against a live SUT. Output: a run report with evidence.

→

/automate

Turn that validated result into an intent spec + a permanent regression pack.

→

/report-bug

When triage finds a real defect, file a structured Bug — human-gated.

```
mock-file:SHOP-456  →  specs/SHOP-456-bulk-discount.md  →  packs/SHOP-456-discount/  —  the full, runnable entry flow.
```

COMMAND · /VALIDATE

Validate a ticket against a live SUT

08

The manual-validation leg. Its output — **a run report with evidence** — is the source of truth a `/automate` run turns into an automated regression.

1 Setup
Read policies + the active SUT's knowledge; fetch & normalize the ticket; triage depth.

2 Environment
Select an env from the manifest; collect version info; mask secrets.

3 Validation
Each AC becomes a check — REST **or** UI. UI runs **headed** so a QA person watches.

4 Write results
Record outcomes back on the ticket — approval-gated.

5 Transition
Move the ticket forward only if every check passes.

6 Report
Save a run report to `runs/<TICKET>_<TS>.md` and append a learning.

COMMAND · /AUTOMATE

09

Turn the validated result into a permanent pack

REST-first, UI fallback — an intent spec plus a regression pack, validated against the gate. **The surface follows the verification.**

0	1	2	2b	3	4	5
Orient	Gather	Spec	Plan	Implement	Validate	Record

Spec = what & why

Context, requirements, a checklist of testable acceptance criteria, persona coverage — **human-approved**.

Plan = how

A REST mapping table, facade shape, persona marker, runtime expectations — **agent-iterated**.

THE SPINE OF THE FRAMEWORK

Never weaken a test to turn it green

10

> **The validation objective never changes during Phase 4. Every fix preserves the acceptance criteria verbatim.**

Forbidden without re-approval

Relax `==` `100` to `>=` `90` · drop a persona marker · add an ungated `xfail` · raise a timeout to silence a real regression.

Loop budget

After **3 cycles** on the same root cause — **STOP and escalate**. The panel mirrors it: twice the same finding → stop.

THE SUT PLUGIN MODEL

The product under test is a plugin

11

A **“site”** under **`sut/<name>/`** owns both its backend access and its tests. The engine depends only on this contract, **never on a specific product** — you wire in **your own** product as a plugin. The two SUTs in the repo are **worked examples**.

manifest.json declares

`source` (path · **`repo`** · ref) · `tests` · `runtime` (in_process / remote) · `env` · `creds` · `knowledge`.

Real source, kept fresh

Point `source.repo` at the backend and `make sync-source` keeps a **local clone** of the real code current — for design, diagnosis and citations.

A second, differently-shaped site drops in with no change to engine/ or policies/ — that is what proves the seam is genuinely generic.

PER-SUT KNOWLEDGE

skills vs learnings — loaded on demand

12

`knowledge.skills`

Authored, standing domain knowledge — what the product does, where contracts live, the gotchas. e.g. `SHOP.md`, `BOOKER.md`.

`knowledge.learnings`

Append-only, dated, AI-accumulated patterns — one captured per Phase-4 cycle as the framework works.

Loaded on demand by `--sut` — only the **active** product's knowledge enters context.

TWO EXAMPLE SUTS SHIP WITH THE FRAMEWORK — DEMOS THAT PROVE THE SEAM, NOT REAL PRODUCTS

`mock-shop` — example · in_process · REST-only · 10% off at ≥3 items

`restful-booker` — example · plugin.py + ui-packs + a live remote env · HTTP 409 no-double-booking

THE REVIEW PANEL

Seven advisory lenses

13

judge
Presides — BLOCK / FIX / FLAG / ESCALATE; writes the human digest.

r-diagnosis
TEST_BUG vs REAL_BUG — before any fix, with cited evidence.

r-evidence
The skeptic. **The green dot is NOT evidence** — pull the raw state.

r-mechanism
SUT-behaviour claims — cite `source:line` or escalate.

r-fidelity
Catches edits that **weaken** an AC to turn a red test green.

r-coverage
Flags an AC the pack never exercises, or a `covers/contract_claim` naming no real rule.

r-uplift
Migration-only — did a ported legacy test adopt these patterns?

The panel raises the floor — it never lowers the bar, and it never blocks a merge. Read-only is tool-enforced (gate: false).

HOW THE REPO IS LAID OUT

One core, many plugins

14

```
commands/    # slash commands — the human-in-the-loop legs
agents/      # the advisory review panel (read-only)
policies/    # 8 product-neutral QA governance policies
engine/      # the deterministic core — owns "green"
sut/         # the SITES under test — plug in YOUR product
  ├─ mock-shop/      # example plugin (demo)
  └─ restful-booker/ # example plugin (demo)
ticket/      # tracker-agnostic ticket contract
docs/        # overview + architecture + gates
tests/       # site-integration bridge (pytest)
```

Two plugin seams

Backend access (`sut/contract.md`) and ticket access (`ticket/contract.md`).

Swap either seam

…and **nothing** in `engine/` or `policies/` changes.

QENSEI, IN ONE LINE

One engine · a plugin per product · a panel that raises the floor

One backend-aware QA engine with **three capabilities** over a single backend connection, the product under test wired in as a **plugin**, an **advisory panel** that raises the floor — and a **deterministic gate** that owns "green."

START HERE

```
$ make demo
$ make demo-booker
$ make design
$ make test
```

CALEVI
Quality assured